สรุปและวิเคราะห์ผล ETL Tool Comparison: 4 Tools สรุปผลการทดสอบ ETL

โปรเจคนี้นำเครื่องมือ 4 อย่างมาทดสอบด้วยข้อมูลการตรวจจับการทุจริตบัตรเครดิต (Credit Card Transactions Fraud Detection) โดยแบ่งขนาดข้อมูลออกเป็นสามระดับ คือ **10,000 แถว 100,000 แถว และ 1,000,000 แถว** แล้วจับเวลาว่าแต่ละเครื่องมือใช้เวลานานแค่ไหนในแต่ละขั้นตอน เครื่องมือที่ใช้เวลาน้อยกว่าถือว่าดีกว่าในการทดสอบนี้

1. ภาพรวม: แต่ละเครื่องมือใช้เวลารวมเท่าไร

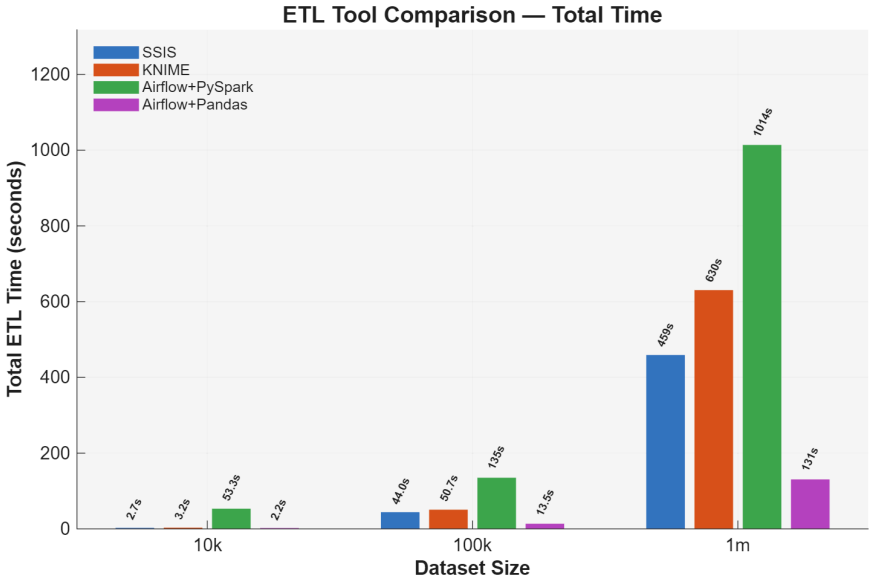

Fig 1 : เวลารวมทั้งหมดของแต่ละเครื่องมือ แยกตามขนาดข้อมูล

กราฟนี้แสดงเวลารวมทั้งหมดของแต่ละเครื่องมือ แกนนอนคือขนาดข้อมูลแบ่งเป็นสามกลุ่ม แต่ละกลุ่มมีแท่งสี่แท่งเรียงกัน แท่งที่สูงกว่าหมายความว่าใช้เวลานานกว่า จากตาราง **Airflow+Pandas เร็วที่สุดในทุกขนาด** โดยที่ข้อมูล 1 ล้านแถวก็ยังใช้เวลาเพียง 131 วินาที ขณะที่ SSIS ใช้ 459 วินาที KNIME ใช้ 630 วินาที และ Airflow+PySpark ช้าที่สุดด้วยเวลาถึง 1,014 วินาที หรือกว่า 16 นาที สิ่งที่น่าสังเกตคือ Airflow+PySpark ช้าตั้งแต่ข้อมูลขนาดเล็กมาก แม้แต่ข้อมูล 10,000 แถว ก็ยังใช้เวลาถึง 53 วินาที ทั้งที่เครื่องมืออื่นทำเสร็จภายใน 2–3 วินาที สาเหตุคือทุกครั้งที่รัน PySpark มันต้องเปิดระบบ Java ขึ้นมาก่อน ซึ่งกินเวลาประมาณ 20–30 วินาทีทุกครั้งโดยไม่ขึ้นกับว่ามีข้อมูลมากหรือน้อยแค่ไหน ทำให้ข้อมูลน้อยๆ เสียเปรียบอย่างชัดเจน

2. สัดส่วนของแต่ละ Phase ต่อเวลารวม - แสดงถึงเวลาหายไปอยู่ในขั้นตอนไหนมากที่สุด

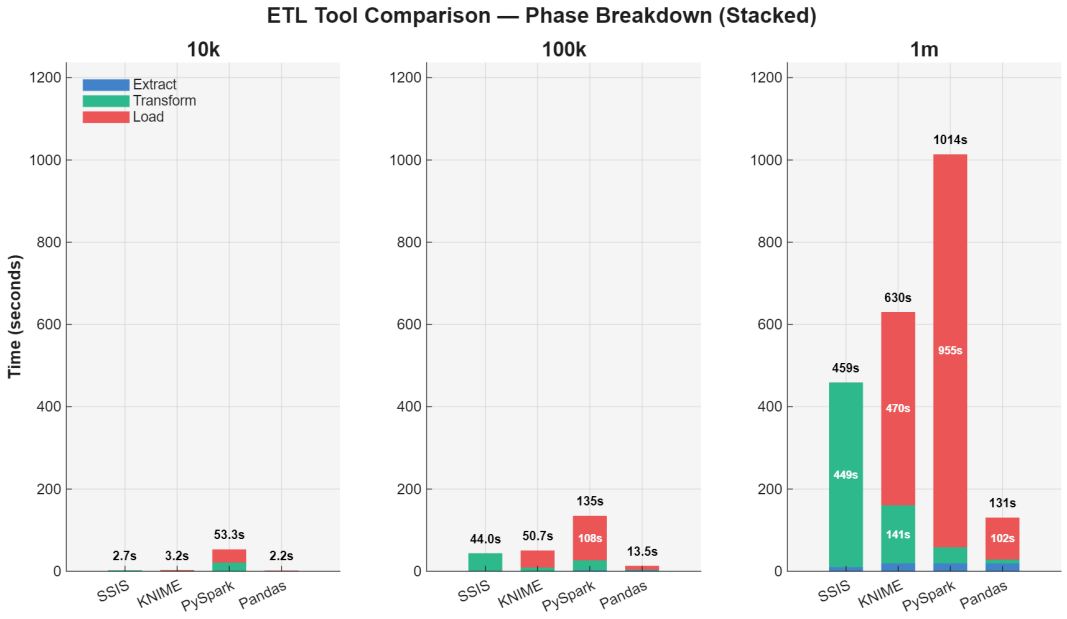

Fig 2 : สัดส่วนของแต่ละ Phase ต่อเวลารวม - แสดงถึงเวลาหายไปอยู่ในขั้นตอนไหนมากที่สุด

กราฟนี้แบ่งแต่ละแท่งออกเป็นสามส่วนซ้อนกันตามสี **สีน้ำเงินคือขั้นดึงข้อมูล(Extract) สีเขียวคือขั้นแปลงข้อมูล(Transform) สีแดงคือขั้นตอนบันทึกข้อมูล(load)** ส่วนที่ใหญ่ที่สุดในแท่งคือขั้นตอนที่กินเวลามากที่สุดของเครื่องมือนั้น ซึ่งสามารถสรุปได้ดังนี้:

SSIS : ขั้นตอน Transform กินเวลาไปเกือบทั้งหมด

ที่ข้อมูล 1 ล้านแถว SSIS ใช้เวลาแปลงข้อมูล(Transform)ไปถึง 449 วินาที จากเวลารวมทั้งหมด 459 วินาที สามารถคิดด้วยวิธีคิดนี้

วิธีคิด: 449 ÷ 459 × 100 = 97.8%

ดังนั้นคิดเป็น 97.8% ของเวลาทั้งหมด ส่วนที่เหลืออีก 2.2% เป็นการดึงและการโหลดข้อมูลรวมกัน

KNIME : ขั้นตอน Load ข้อมูลคือจุดที่ช้าที่สุด

ที่ข้อมูล 1 ล้านแถว KNIME ใช้เวลาบันทึกข้อมูลไป 470 วินาที จากเวลารวม 630 วินาที คิดเป็น 74.6%

วิธีคิด: 470 ÷ 630 × 100 = 74.6%

สาเหตุคือ KNIME ในขั้นตอนโหลดข้อมูลมีการใช้ DB Writer Node ทำให้ข้อมูลถูกส่งทีละ 1 แถว MySQL ต้องบันทึกและยืนยันทุกคำสั่ง ทำให้เวลา Load พุ่งสูง

Airflow+PySpark : ขั้นตอน Load ข้อมูลคือจุดที่ช้าที่สุดในทุกเครื่องมือ

ที่ข้อมูล 1 ล้านแถว PySpark ใช้เวลาบันทึกข้อมูลไปถึง 955 วินาที จากเวลารวม 1,014 วินาที คิดเป็น 94.2%

วิธีคิด: 955 ÷ 1,014 × 100 = 94.2%

ปัญหาเดียวกับ KNIME คือส่งข้อมูลทีละแถวผ่านช่องทางเดียว แต่รุนแรงกว่าเพราะไม่ได้ตั้งค่าการกระจายงาน ทำให้ข้อมูลทั้งหมดถูกส่งออกผ่าน JDBC เพียงช่องทางเดียว ทำให้เวลาพุ่งสูงกว่าทุกเครื่องมือ

Airflow+Pandas : สัดส่วนทุกขั้นตอนพอสมดุล

ที่ข้อมูล 1 ล้านแถว Pandas ใช้เวลาบันทึกข้อมูลไป 102 วินาที จากเวลารวม 131 วินาที คิดเป็น 78.1%

วิธีคิด: 102 ÷ 131 × 100 = 78.1%

Pandas ส่งข้อมูลเป็นกลุ่มใหญ่ทีเดียวแทนที่จะส่งทีละแถว ทำให้ขั้นบันทึกข้อมูลใช้เวลาน้อยกว่าเครื่องมืออื่นมาก

---

3. เมื่อข้อมูลโตขึ้น จะเวลามากขึ้นแค่ไหน

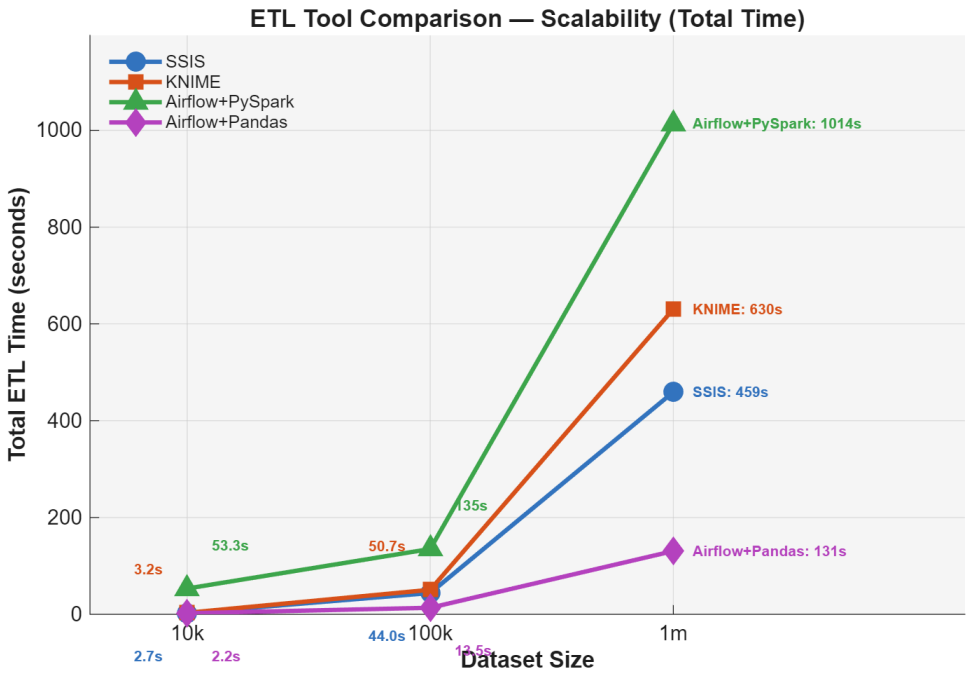

Fig 3 : เมื่อข้อมูลโตขึ้น จะเวลามากขึ้นแค่ไหน

เส้นของ Airflow+PySpark ชันที่สุดอย่างเห็นได้ชัด จากจุดเริ่มต้นที่ 53 วินาทีที่ข้อมูล 10,000 แถว พุ่งขึ้นไปถึง 1,014 วินาทีที่ข้อมูล 1 ล้านแถว เพิ่มขึ้น 19 เท่า เส้นของ SSIS และ KNIME มีความชันใกล้เคียงกัน ทั้งคู่เริ่มต้นต่ำแต่เวลาก็โตเร็วเมื่อข้อมูลใหญ่ขึ้น เส้นของ Airflow+Pandas ราบเรียบที่สุด โตจาก 2.2 วินาทีเป็น 131 วินาที คิดเป็น 60 เท่า ซึ่งน้อยที่สุดในบรรดาเครื่องมือที่เริ่มต้นด้วยเวลาต่ำ

ช่วงที่ข้อมูลโต	SSIS	KNIME	PySpark	Pandas
10k → 100k (ข้อมูลโต 10 เท่า)	ใช้เวลาเพิ่มขึ้น 16 เท่า	ใช้เวลาเพิ่มขึ้น16 เท่า	ใช้เวลาเพิ่มขึ้น 2.5 เท่า	ใช้เวลาเพิ่มขึ้น 6 เท่า
100k → 1m (ข้อมูลโต 10 เท่า)	ใช้เวลาเพิ่มขึ้น 10 เท่า	ใช้เวลาเพิ่มขึ้น 12 เท่า	ใช้เวลาเพิ่มขึ้น 7.5 เท่า	ใช้เวลาเพิ่มขึ้น 10 เท่า
10k → 1m (ข้อมูลโต 100 เท่า)	ใช้เวลาเพิ่มขึ้น **172** เท่า	ใช้เวลาเพิ่มขึ้น **200** เท่า	ใช้เวลาเพิ่มขึ้น **19** เท่า	ใช้เวลาเพิ่มขึ้น 60 เท่า

จากตารางในแถวที่ 10k → 1m เป็นข้อมูลโต 100 เท่าจะสังเกตได้ว่า PySpark ใช้เวลาเพิ่มขึ้น **19** เท่าซึ่งถ้าเทียบกับเครื่องมืออื่นๆ ถือว่าใช้เวลาน้อยที่สุดแต่ต้องคำนึงด้วยว่า PySpark จุดเริ่มต้น คือ 53 วินาที หากเทียบกับเครื่องมืออื่นๆ ที่มีจุดเริ่มต้นน้อยกว่า เช่น หากเปรียบเทียบกับ Pandas เริ่มที่ 2.2 วินาที ดังนั้น PySpark จึงยังช้ากว่า Pandas ตลอดทุกขนาด แม้อัตราการโตจะดูดีกว่าก็ตาม

4. การเปรียบเทียบเวลาในการประมวลผล ETL ตามขนาดข้อมูลในสเกลลอการิทึม

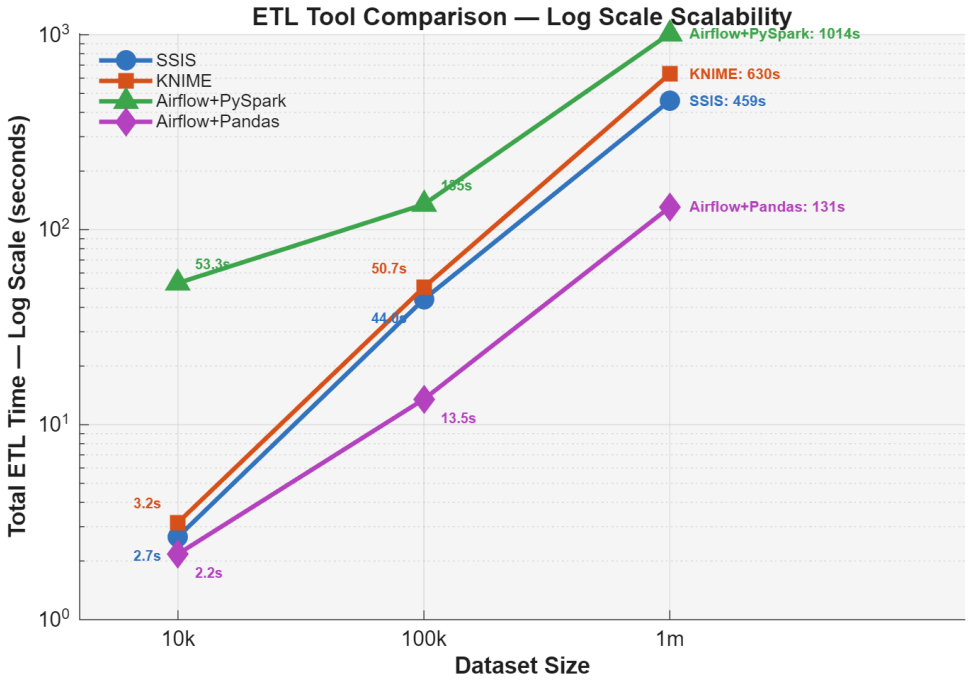

Fig 4 : การเปรียบเทียบเวลาในการประมวลผล ETL ตามขนาดข้อมูลในสเกลลอการิทึม

ปัญหาของกราฟที่ 3 คือ เมื่อ PySpark สูงถึง 1,014 วินาที เส้นของ Pandas ที่ 131 วินาที จะแทบแนบติดกับแกนล่าง ทำให้มองไม่เห็นรายละเอียด กราฟที่ 4 แก้ปัญหานี้ด้วยการปรับแกน Y ให้แต่ละช่วงกระโดดขึ้น 10 เท่าแทนที่จะเพิ่มขึ้นทีละเท่ากัน

วิธีอ่านกราฟนี้ต่างจากกราฟที่ 3 ตรงที่ระยะห่างในแนวตั้งไม่ได้วัดว่าต่างกันกี่วินาที แต่วัดว่าต่างกันกี่เท่า เส้นที่อยู่สูงกว่าในกราฟนี้ยังหมายถึงใช้เวลานานกว่าเสมอ และเส้นที่ชันกว่าหมายถึงโตเร็วกว่าเมื่อข้อมูลเพิ่มขึ้น

ผลที่เห็นได้ชัดจากกราฟนี้คือ เส้นของ Airflow+Pandas อยู่ต่ำสุดตลอดและความชันค่อนข้างคงที่ เส้นของ SSIS และ KNIME เกือบทับกันตลอด แสดงว่าพฤติกรรมการโตของเวลาคล้ายกันมาก และเส้นของ PySpark อยู่สูงสุดตลอดและยิ่งชันขึ้นในช่วง 100k ถึง 1m ซึ่งบอกว่ายิ่งข้อมูลมากขึ้น PySpark ยิ่งช้าขึ้นในอัตราที่เร็วขึ้นด้วย

บทสรุป

จากการทดสอบทั้งหมด **Airflow+Pandas ชนะในทุกขนาดข้อมูล** เพราะใช้วิธีเรียบง่ายที่เหมาะกับการทำงานบนเครื่องเดียว คือส่งข้อมูลเป็นกลุ่มใหญ่และไม่ต้องเปิดระบบพิเศษก่อนทำงาน

SSIS เร็วในขั้นดึงและบันทึกข้อมูล(Load) แต่ช้าในขั้นแปลงข้อมูล(Transform) เพราะทำทีละแถวและไม่สามารถเขียนทับคอลัมน์เดิมได้โดยตรง ต่างจาก Python และ KNIME ที่

มองข้อมูลเป็นตารางทั้งก้อน จึงแก้ไขค่าในคอลัมน์เดิมได้ทันทีในคำสั่งเดียว KNIME ดึงข้อมูลได้เร็วที่สุดในทุกขนาดแต่บันทึกข้อมูล(Load)ช้าเพราะส่งทีละแถวเช่นกัน ทั้งสองเครื่องมือนี้การใช้เวลาใช้การประมวลผลในแต่ละขั้นตอนที่คล้ายกัน

Airflow+PySpark ช้าที่สุด เพราะออกแบบมาสำหรับการทำงานบนหลายเครื่องพร้อมกัน เมื่อใช้บนเครื่องเดียวจึงยังไม่แสดงศักยภาพจริง การเปิดระบบ Java และการบันทึกข้อมูล(Load)ทีละแถวรวมกันทำให้ช้ากว่าทุกเครื่องมือในทุกขนาดที่ทดสอบ

สมมติฐานกับผลจริง

ผลที่ได้จากการทดสอบแตกต่างจากที่คาดไว้เกือบทุกแนวทาง กล่าวคือ

SSIS ที่คาดว่าจะเร็วที่สุด ผลจริงได้อันดับที่ 2 เพราะขั้นตอนแปลงข้อมูลใช้เวลานานกว่าที่คาด

KNIME ที่คาดว่าจะช้าที่สุด ผลจริงได้อันดับที่ 3 ไม่ใช่ช้าที่สุดอย่างที่คาด

Airflow ร่วมกับ PySpark ที่คาดว่าจะอยู่ในระดับกลาง ผลจริงกลับช้าที่สุดในทุกขนาดข้อมูล

Airflow ร่วมกับ pandas ที่คาดว่าจะใกล้เคียงกับ PySpark ผลจริงกลับเร็วที่สุดในทุกขนาดข้อมูล

จากผลที่ได้ทำให้กลุ่มนักศึกษาผู้จัดทำได้บทเรียนว่า เครื่องมือที่ดูซับซ้อนหรือทรงพลังที่สุดไม่ได้หมายความว่าจะเร็วที่สุดเสมอไป ขึ้นอยู่กับว่าใช้กับข้อมูลขนาดใดและในสภาพแวดล้อมแบบใดด้วย